

CAS4160 Homework 7: RLHF

[Kim Minsung]
[2020142087]

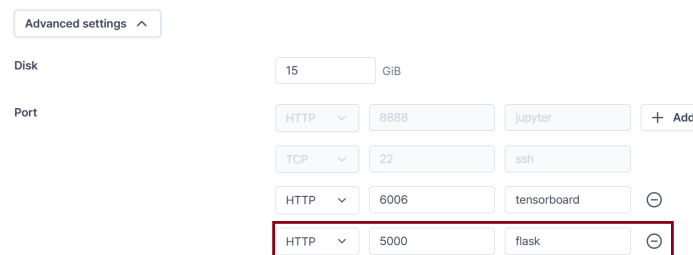
1 Introduction

The goal of this assignment is to understand and gain hands-on experience with Reinforcement Learning from Human Feedback (RLHF). Unlike traditional reinforcement learning algorithms that rely on predefined environment rewards, RLHF learns a reward model from human feedback, and then uses that learned reward to train a policy.

While RLHF is widely known for its use in training large language models (LLMs), the original research was actually conducted in the context of robot control.

In this assignment, you will implement an RLHF framework by extending the PPO algorithm we previously developed. Instead of using an environment-provided reward, you will define a reward function based on human preferences, and train this reward model accordingly. You will then provide feedback directly to guide the robot's behavior, giving you a practical understanding of how RLHF works and having fun to see robot evolving.

2 Installation



The screenshot shows the 'Advanced settings' interface of VESL AI. It includes a 'Disk' section with a 15 GIB value and a 'Port' section with a table of open ports. The table has columns for protocol, port number, and application. The 'flask' entry is highlighted with a red box.

Protocol	Port	Application	Action
HTTP	8888	jupyter	+ Add
TCP	22	ssh	
HTTP	6006	tensorboard	⊖
HTTP	5000	flask	⊖

Figure 1: Open port 5000 for Flask

In this assignment, unlike previous ones, you will need to install additional packages. Please make sure to check the `requirements.txt` file and install all required dependencies before starting.

Additionally, to provide feedback (preference), we will use a web framework called Flask. If you are using VESL AI, make sure to open additional ports to allow the web interface to function properly. Please refer to Figure 1 for more detail. Please use the Chrome browser for this.

3 Code structure overview

The code structure is similar to that of the HW2, but we are going to add few more parts for RLHF. In this assignment, you mainly work with the following codes:

- `cas4160/networks/reward_predictor.py`: Reward model implementation.
- `cas4160/agents/pg_agent.py`: PPO agent with the learned reward model.
- `cas4160/scripts/run_hw7.py`: The main training loop for your implementation.

4 Reinforcement learning from human feedback

We will provide a brief overview of Reinforcement Learning from Human Feedback (RLHF), followed by implementation. You will: (1) define a reward function and implement the reward learning objective, (2) collect human-annotated preference samples, store them in a replay buffer, and train the reward model, and finally, (3) use the learned reward to train a PPO agent.

4.1 Formulation of preference

We consider an agent interacting with an environment over a sequence of steps. At each timestep t , the agent observes a state $s_t \in \mathcal{S}$ and selects an action $a_t \in \mathcal{A}$. In standard RL, the environment returns a scalar reward $r_t \in \mathbb{R}$, and the agent's goal is to maximize the expected cumulative discounted reward.

In contrast, we assume that the environment does not provide explicit rewards. Instead, a human overseer gives *preference comparisons* between short trajectory segments. A trajectory segment is defined as a sequence:

$$\tau = ((s_0, a_0), (s_1, a_1), \dots, (s_{k-1}, a_{k-1})).$$

We denote $\tau_1 \succ \tau_2$ to indicate that the human prefers segment τ_1 over τ_2 .

Goal. The agent should generate trajectories preferred by the human, while minimizing the number of preference queries.

Evaluation criteria.

- **Quantitative evaluation:** Assume preferences are induced by an unknown reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. Then $\tau_1 \succ \tau_2$ implies:

$$\sum_{t=0}^{k-1} r(s_t^1, a_t^1) > \sum_{t=0}^{k-1} r(s_t^2, a_t^2).$$

If r is known, we can evaluate agents based on their true cumulative rewards.

- **Qualitative evaluation:** When r is unknown, we evaluate agents based on how well their behavior aligns with human preferences. This involves presenting a natural language goal to a human and asking for evaluations based on observed agent behavior videos.

In our setup, the human overseer is presented with a visualization of two trajectory segments in the form of short movie clips, each lasting between 1 to 2 seconds.

The human annotator provides feedback by indicating one of the following:

- τ_1 is preferred than τ_2 .
- τ_2 is preferred than τ_1 .
- Both segments are equally good.

Each comparison is stored as a triplet (τ_1, τ_2, μ) in a replay buffer $\mathcal{D}$, where:

- τ_1 and τ_2 are the two compared trajectory segments,
- μ is a probability distribution over $\{1, 2\}$ representing the user’s preference.
- If the user prefers one segment, μ assigns all mass to that segment.
- If both segments are marked as equally preferable, then $\mu = (0.5, 0.5)$.

4.2 Fitting reward function

The estimated reward function $\hat{r}$ can be considered as a latent variable explaining human judgments, a **preference function** $\hat{P}$. We model the probability that a human prefers a trajectory segment τ_1 over τ_2 as:

$$\hat{P}[\tau_1 \succ \tau_2] = \frac{\exp(\sum \hat{r}(\mathbf{s}_t^1, \mathbf{a}_t^1))}{\exp(\sum \hat{r}(\mathbf{s}_t^1, \mathbf{a}_t^1)) + \exp(\sum \hat{r}(\mathbf{s}_t^2, \mathbf{a}_t^2))}, \quad (1)$$

where the summation is taken over all timesteps within each segment.

To train $\hat{r}$, we minimize the cross-entropy loss between this predicted preference and the actual human-labeled preference distribution μ :

$$\mathcal{L}(\hat{r}) = - \sum_{(\tau_1, \tau_2, \mu) \in \mathcal{D}} \left[\mu(1) \log \hat{P}[\tau_1 \succ \tau_2] + \mu(2) \log \hat{P}[\tau_2 \succ \tau_1] \right] \quad (2)$$

4.3 Implementation

Let’s implement RLHF step by step:

1. `cas4160/networks/reward_predictor.py`: Implement all TODOs in this file to train a reward model.
2. `cas4160/agents/pg_agent.py`: This code implements PPO, but you need to make it to use the reward function implemented above by changing only one line of code.
3. `cas4160/scripts/run_hw7.py`: You need to implement to save human annotated triplets to a replay buffer and to use it for reward model training, and to provide reward signals to the PPO agent. Implement all TODOs in this file.

5 Experiments

5.1 PointMaze with synthetic preferences

- This task is designed to help debug your code and understand RLHF and feedback mechanisms. In the maze, there is a green ball and a red target. The goal is for the green ball to move toward the red target. You should provide feedback by “preferring” the trajectory that appears more likely to reach the target based on the green ball’s motion.
- Let’s first train the reward model using synthetic preferences. Instead of receiving human feedback, the environment’s reward is used to generate synthetic preferences, which are then used for training. Take a look at how this works in `run_hw7.py` when `syn_prefs` is set to **True**. Run the bash file below to train a model at `PointMaze_UMazeDense-v3` on synthetic preferences.

```
sh pointmaze_syn_prefs.sh
```

- After training is complete, open TensorBoard to check the training results. Note that you can also download rollout gifs from tensorboard.
- **Question:** Check the training results and evaluation rollouts. How did the trained agent perform? Explain why the results are not optimal, based on the reward, and the structure of the maze. And attach the rollout frames at Figure 2.

Answer:

Add your explanation.

The trained agent partially learned the intended behavior, but the result was not fully successful. As shown in Figure 2, the green ball moves from the initial position toward the upper part of the maze and becomes closer to the red target than at the beginning of the rollout. This indicates that the learned reward model captured some useful preference information from the synthetic comparisons. However, the agent does not completely reach the red target. Instead, its behavior seems to plateau near the upper-left corridor region, where it is closer to the goal in terms of distance but still separated from the target by the maze structure.

The main reason for this suboptimal behavior is that the synthetic preferences are generated from the environment reward, which mainly reflects how close the agent is to the target. In PointMaze, however, directly reducing the Euclidean distance to the goal is not always equivalent to solving the task. Because the maze contains walls and a U-shaped passage, the agent cannot simply move straight toward the red target. It must first follow the valid corridor and then approach the goal through the correct opening. Therefore, a reward signal based mostly on local distance-to-goal information can encourage the agent to move toward positions that appear close to the goal geometrically, even if those positions do not lead to a feasible path to the target.

Another limitation is that the preference labels are generated by comparing short trajectory segments, and in the synthetic setting the comparison is based on the final reward value of the segment. This makes the learned reward model focus more on which segment ends closer to the goal, rather than whether the overall trajectory follows a globally correct path through the maze. As a result, the reward model can provide useful but imperfect guidance to PPO. The policy improves compared to random behavior, but it may still get stuck in a locally attractive region and fail to complete the full navigation task.

Overall, the synthetic preference experiment shows that the RLHF pipeline is functioning: trajectory comparisons are collected, the reward model learns a preference-based reward, and PPO uses this learned reward to improve the policy. However, the learned behavior is not optimal because the synthetic reward does not fully capture the long-horizon structure of the maze. A better preference signal would need to evaluate whether the agent follows the correct route through the corridor, not only whether it becomes locally closer to the target.

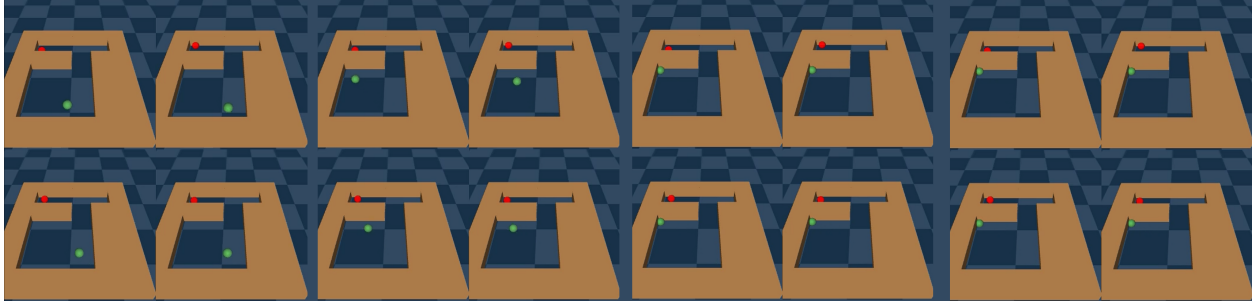


Figure 2: PointMaze rollout with synthetic preferences. The green ball moves closer to the red target compared to the initial state, but it does not fully reach the goal. The policy partially learns goal-directed behavior, while still suffering from the local-distance limitation of the synthetic preference signal.

5.2 PointMaze with human preferences

- This time, let's solve the maze more effectively using your feedback. This bash script runs the code that includes the trajectory annotation GUI.

```
sh pointmaze.sh
```

- If you launch the bash file, you can see these outputs from terminal:

```
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://***.***.***.***:5000
Press CTRL+C to quit
Using GPU id 0

***** Iteration 0 *****

Collecting video rollouts for feedback...
```

This means your training run has started. Now, you can provide preference feedback via your web browser at 127.0.0.1:5000. Note that if you are using VESSL AI, you should navigate to metadata panel and click port 5000 as you open the port for TensorBoard. Then, the web browser will show you the GUI interface, as shown in Figure 3. You can click buttons or use arrow keys in your keyboard to provide feedback.



Figure 3: Web GUI for human preference feedback.

- Once training is complete, you can use TensorBoard to visualize the results. Since this task is very simple, training should finish in about 10 minutes. If you provide reasonable feedback, you will see the green ball moving smoothly toward the red goal without any errors.

Here are some tips for giving feedback:

- The videos you see are not the whole trajectory of policy rollout. To speed up training, we only show the last few seconds.
- Make some concrete criteria internally and give consistent feedback during an experiment.
- Nothing is perfect on the first try. Choose the trajectory that shows even a slight movement toward the goal. It will gradually improve over time.
- Avoid choosing 'Neutral' unless absolutely necessary. Any small differences can lead to big changes. Select 'Neutral' only when it is truly hard to distinguish between the two trajectories.
- If your trained model shows no potential to follow the intended goal, restart the training.
- If the same action was taken, the one performed faster is better.

Question. Please provide the successful rollout frames of your trained agent in Figure 4. Explain why human feedback is beneficial for this task, and discuss the conditions under which RLHF becomes necessary or particularly helpful for policy learning.

Answer:

Add your explanation.

The trained agent successfully learned to solve the PointMaze task with human preferences. As shown in Figure 4, the green ball starts from the lower-left region of the maze, moves through the valid corridor, and eventually reaches the red target. Unlike the synthetic preference experiment, the policy does not simply move toward a locally closer position. Instead, it learns a more task-relevant behavior: following the feasible path around the wall structure and approaching the goal through the correct corridor.

Human feedback is beneficial in this task because the desired behavior is not fully captured by a simple local reward such as Euclidean distance to the target. In a maze environment, the shortest geometric direction

toward the goal may be blocked by walls. Therefore, an agent that only receives a distance-based signal can be encouraged to move toward states that are close to the target in coordinate space but are not actually on a successful path. Human feedback can provide higher-level preference information by judging whether a trajectory is making meaningful progress along the correct route, rather than only whether the final position is numerically closer to the goal.

In this experiment, I gave preferences based on whether the green ball moved through the valid passage toward the red goal, whether it avoided getting stuck near walls, and whether it reached the target faster when both trajectories were similar. These preference comparisons allowed the reward predictor to learn a reward function that better matched the intended task objective. Then, PPO used this learned reward to improve the policy, resulting in a rollout that reaches the goal.

RLHF becomes necessary or particularly helpful when designing a hand-crafted reward function is difficult, ambiguous, or likely to produce unintended behavior. This includes tasks with sparse rewards, long-horizon planning, obstacles, qualitative goals, or behaviors that are easy for humans to recognize but hard to express mathematically. For example, in PointMaze, humans can easily tell which trajectory follows the correct corridor toward the goal, but a simple reward based only on distance can be misleading. Thus, RLHF is especially useful when human judgment can provide more accurate task guidance than an imperfect manually designed reward.

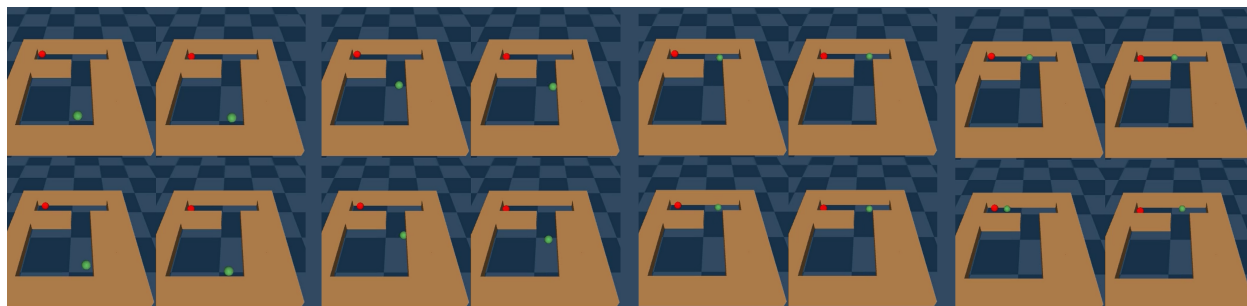


Figure 4: PointMaze rollout with human preferences. The green ball successfully follows the valid corridor and reaches the red target, showing that human preference feedback helped the learned reward model capture the intended navigation behavior.

5.3 Hopper Backflip

Alright. But, it is not too surprising. Shall we try something a bit more interesting? In this section, we will use the Hopper environment, where we have tested various algorithms before, to attempt a backflip. Imagine trying to design a reward function that makes Hopper perform a backflip—sounds tough, right? But with RLHF, just a bit of your quality feedback and a little more time is all it takes to get Hopper to do a backflip!

- Train a model on Hopper-v5. This bash script changes to the execution directory and runs the code that includes the trajectory annotation GUI.

```
sh hopper.sh
```

- This experiment performs 100 optimization steps while asking for human feedback every 5 steps. The entire experiment will approximately take 1 hour and 20 minutes in VESSL AI.

- We include *hopper_flip.mp4* in the homework folder to show the example of hopper learning to do a backflip. You can give a feedback based on that video. Show your prompting skills!
- A backflip is considered successful if the hopper completes one backward revolution. Contact with the grounding during the motion is allowed.

Question. Please provide the rollout of your trained policy in Figure 5 by sampling a few frames from the saved video. Plot `Eval_AverageReturn` of your policy to Figure 6. Finally, include the last successful rollout video as `[Student_ID]_Hopper_Backflip.gif` inside the submission zip file.

Explanation:

The trained Hopper policy successfully learned a backflip-like behavior from human preference feedback. As shown in Figure 5, the Hopper first bends its body backward, then uses ground contact to generate rotational momentum. After that, the body rotates backward and passes through a sequence of inverted or near-inverted postures before returning toward the ground. This behavior satisfies the task objective because the goal of this experiment is not normal forward hopping, but completing one backward revolution. Ground contact during the motion is allowed, so the policy can use the floor as support while producing the rotation.

This task is much harder to specify with a hand-designed reward than PointMaze. The default Hopper reward in standard reinforcement learning is usually designed for stable forward locomotion, survival, and smooth movement. However, a backflip requires behavior that is very different from normal locomotion: the agent must intentionally lean backward, lose the usual upright posture, rotate its body, and sometimes make contact with the ground. Therefore, a manually designed reward can easily conflict with the desired behavior or lead to unintended strategies.

Human feedback is useful here because humans can directly compare two short rollout clips and prefer the one that looks more like a backflip. During annotation, useful criteria include whether the Hopper rotates backward, whether it gains enough angular momentum, whether the body becomes inverted, and whether it completes a full backward revolution. These qualitative properties are easy for humans to judge from videos, but difficult to encode precisely as a scalar reward function.

The average return curve in Figure 6 shows the evaluation return of the policy during training. Since the reward used in this RLHF setup is produced by the learned reward predictor, the plotted `Eval_AverageReturn` should be interpreted as the policy's performance under the learned preference-based reward, rather than the original environment reward. An increasing or improving trend indicates that PPO is learning actions that the reward model considers more consistent with the human-preferred backflip behavior. Together with the rollout frames, this suggests that the learned reward model provided a useful training signal for acquiring the backflip policy.

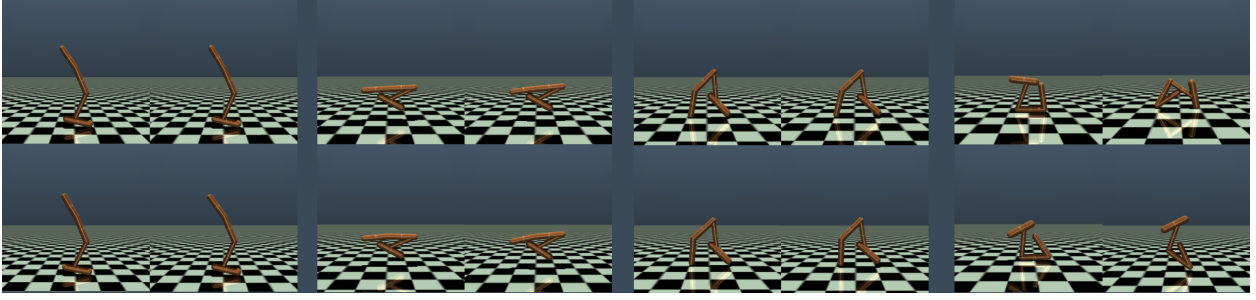


Figure 5: Hopper backflip rollout generated by the trained RLHF policy. The sampled frames show the Hopper bending backward, generating rotational momentum through ground contact, rotating backward, and reaching inverted or near-inverted postures. This demonstrates that the learned policy acquired the desired backflip-like behavior from human preference feedback.

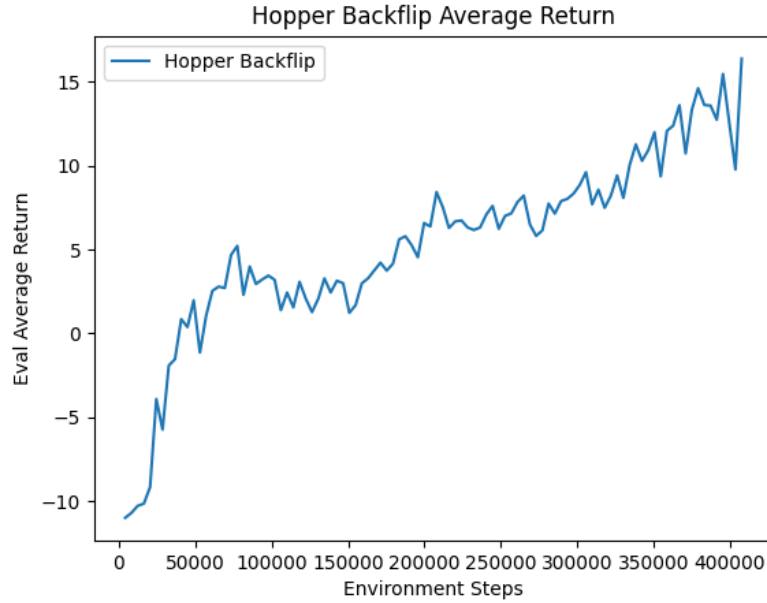


Figure 6: Evaluation average return of the Hopper backflip policy during RLHF training. The return is computed using the learned reward predictor, so it measures how strongly the policy aligns with the preference-based reward model rather than the original Hopper environment reward.

6 Submission

Please submit the code, a backflip gif, and the **“report”** in a single zip file, `hw7_[YourStudentID].zip`. The zip file should be smaller than 50MB. The structure of the submission file should be:

```
hw7_[YourStudentID].zip
├── hw7_[YourStudentID].pdf
├── [YourStudentID]_Hopper_Backflip.gif
├── cas4160
│   ├── agents
│   │   └── pg_agent.py
│   ├── networks
│   │   ├── reward_predictor.py
│   │   └── ...
│   └── ...
└── ...
```

We hope you enjoy this assignment, and thank you for all your hard work throughout the semester. We also hope these homework assignments have deepened your understanding of reinforcement learning!